

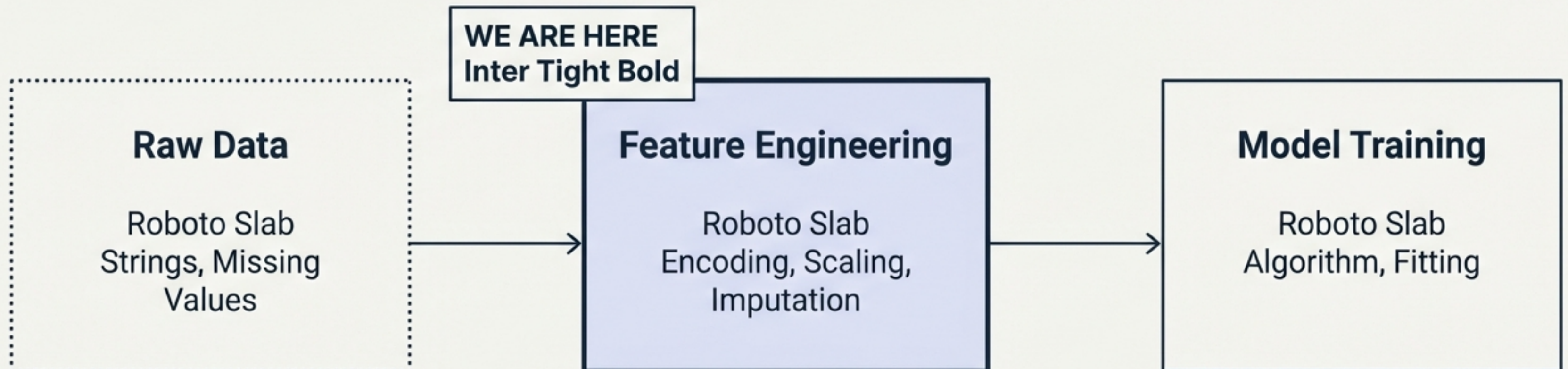
Feature Engineering: Handling Nominal Categorical Data

The theory and mechanics of One Hot Encoding (OHE)

The Goal: Convert raw text categories into numerical inputs for ML algorithms.

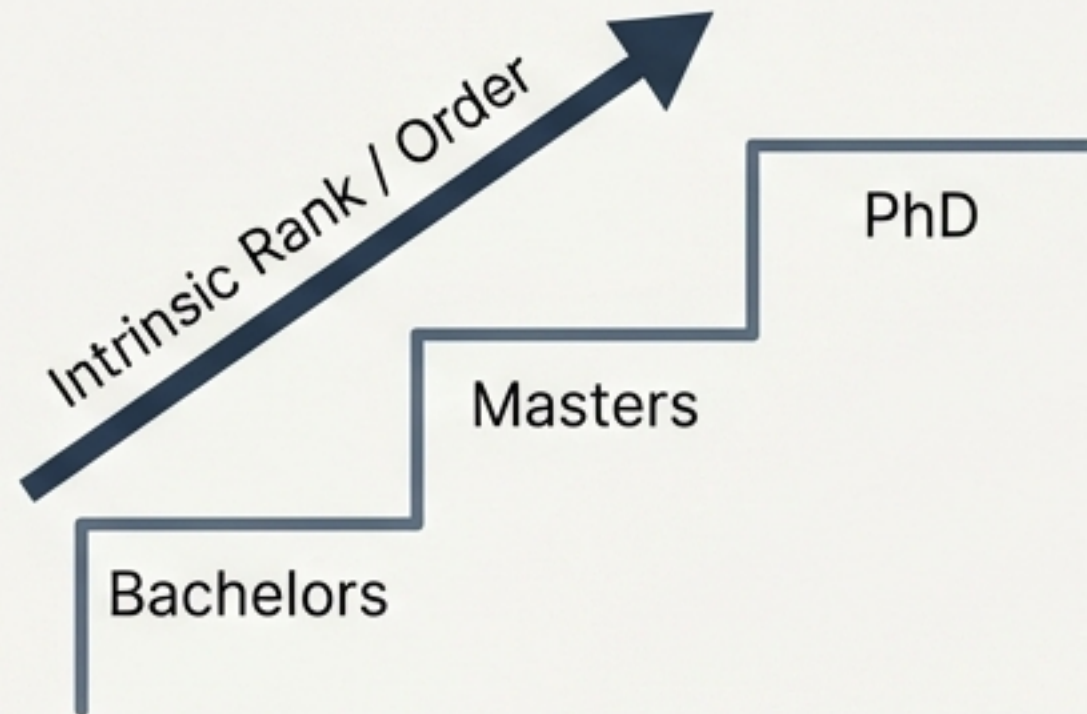
The Challenge: Handling variables like City or Brand without creating mathematical errors.

The Scope: Nominal Data Transformation using One Hot Encoding.



Distinguishing Between Ordinal and Nominal Data

Ordinal Data



Categories possess a mathematical hierarchy. Elements can be compared (Greater Than / Less Than).

Strategy:
Use Ordinal Encoding
(1, 2, 3)

Nominal Data



Categories have no rank. Elements are distinct but equal in status.

Strategy:
Use One Hot Encoding
(Binary Vectors)

Key Insight: Treating Nominal data as Ordinal forces false relationships (e.g., Red > Blue) into the model.

The Risk of Introducing False Hierarchies

Wrong Approach: Ordinal Encoding on Colors

1	Yellow →	0
2	Blue →	1
3	Red →	2

If Red = 2 and Blue = 1...

> • Model infers:
Red > Blue

? • Model infers:
Blue + Blue = Red

The Consequence: Algorithms find patterns that do not exist. Assigning ranked numbers to independent categories creates noise and biases prediction.

Conclusion: We must use a representation where all categories remain mathematically equidistant.

The Solution: One Hot Encoding (OHE)

Input Feature

Colour
Yellow
Blue
Red
Yellow



Output Vectors (Dummy Variables)

Colour_Yellow	Colour_Blue	Colour_Red
1	0	0
0	1	0
0	0	1
1	0	0

Definition: Convert categorical strings into integer vectors. '1' represents presence, '0' represents absence. This maintains independence between categories.

The Obstacle: The Dummy Variable Trap

The Problem of Multicollinearity

Colour_Yellow Colour_Blue Colour_Red

$$\text{Colour_Yellow} + \text{Colour_Blue} + \text{Colour_Red} = 1$$

- **The Issue:** The input columns have a perfect linear relationship. The value of one can always be predicted by the sum of the others.
- **Why it Matters:** Linear models (Linear Regression, Logistic Regression) assume inputs are independent. This dependence creates infinite solutions for coefficients, causing model instability.
- **Term:** This redundancy is known as the **Dummy Variable Trap**.

Escaping the Trap: The N-1 Rule

Colour_Yellow	**Sufficiency Set**	
	Colour_Blue	Colour_Red
1	0	0
0	1	0
0	0	1
1	0	0

The Logic Check:

If `Blue = 0` AND `Red = 0`... Then the car **MUST** be Yellow. We do not need the 'Yellow' column to know it is Yellow. Absence of others implies presence of the first.

Rule: If you have N categories, you only need N-1 columns to represent them fully. Always drop one column (usually the first) to restore independence.

The Scaling Problem: High Cardinality

The Curse of Dimensionality

- **Scenario:** A car dataset with 32 unique brands.
- **Result:** 32 new input columns created.
- **Consequence:**
 - **Aggressive increase** in dataset width (dimensionality).
 - **Slower** training times.
 - **Sparse matrices** (mostly zeros) which degrade performance.

****Brand****

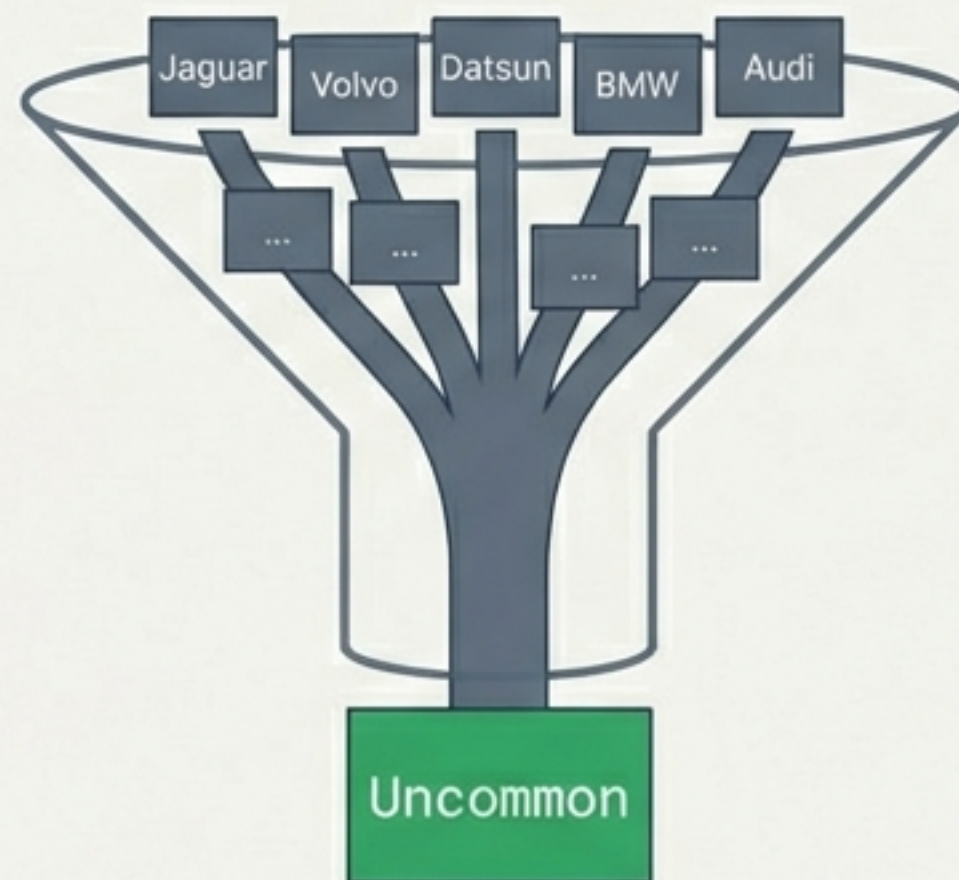
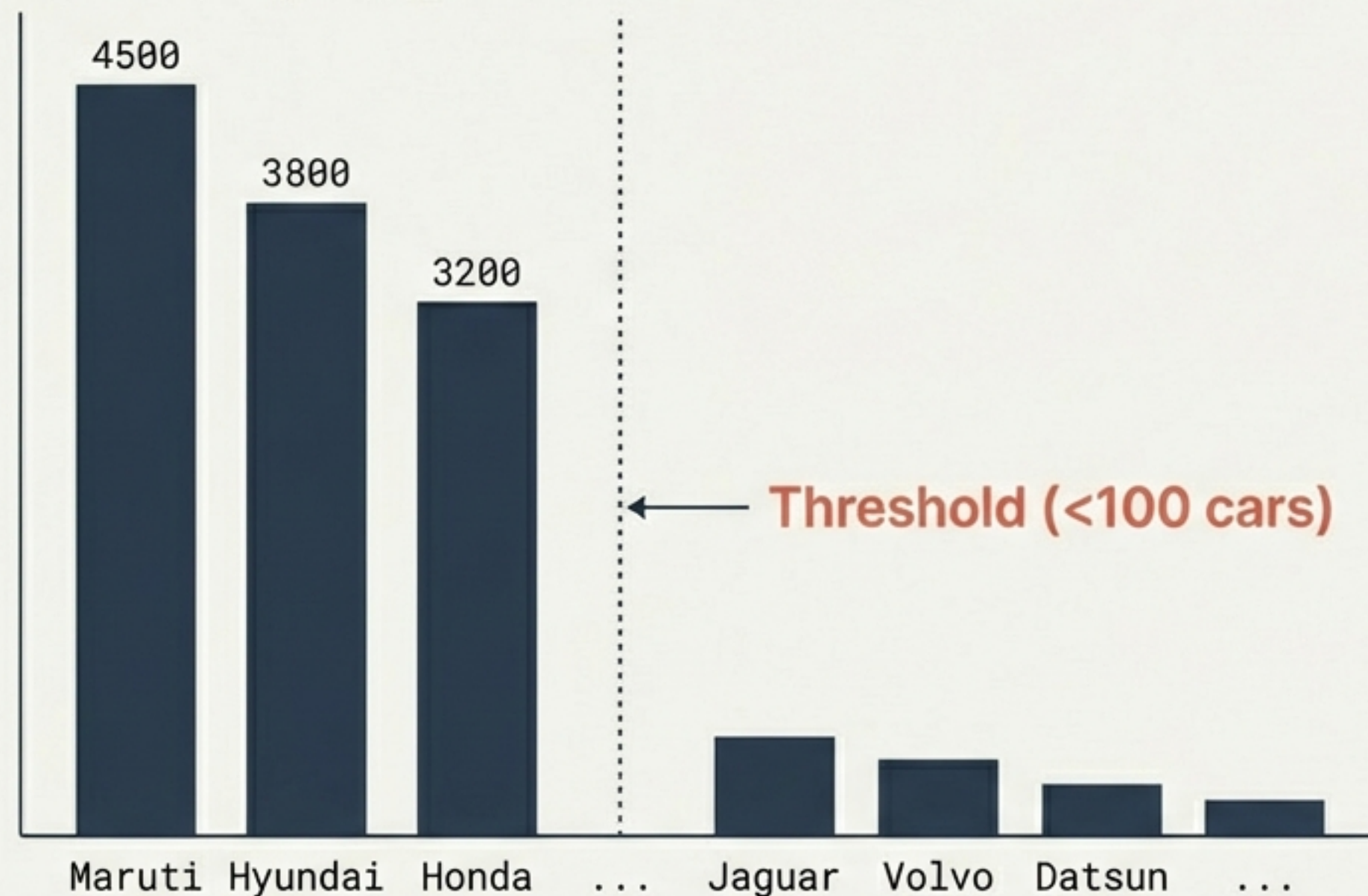
Maruti,
Hyundai,
Skoda,
Jaguar,
Volvo,
Datsun,
BMW,
Audi...
...

OHE

Brand_Maruti	Brand_Hyundai	Brand_Skoda	Brand_Jaguar	Brand_Volvo	Brand_Datsun	Brand_BMW	Brand_Audi	...
JetBrains Mono	JetBrains Mono	JetBrains Mono	JetBrains Mono	JetBrains Mono	JetBrains Mono	JetBrains Mono	JetBrains Mono	...
1	0	0	0	0	0	0	0	...
0	1	0	0	1	0	0	0	...
0	0	0	1	0	1	0	0	...
0	0	1	0	0	0	1	0	...
0	0	0	0	0	0	1	0	...
0	0	1	0	0	0	0	1	...
...	...	...	...	...	...	...	...	...

Managing Dimensions: The 'Uncommon' Grouping Strategy

Brand Frequency



Strategy: Keep frequent categories distinct. Group rare categories into a single 'Other' or 'Uncommon' bucket.

Impact: Reduces dimensionality from 32 columns down to ~12 robust columns.

Implementation: The Pandas Approach

```
import pandas as pd
# One Line Solution
pd.get_dummies(df, columns=['Fuel', 'Owner'], drop_first=True)
```



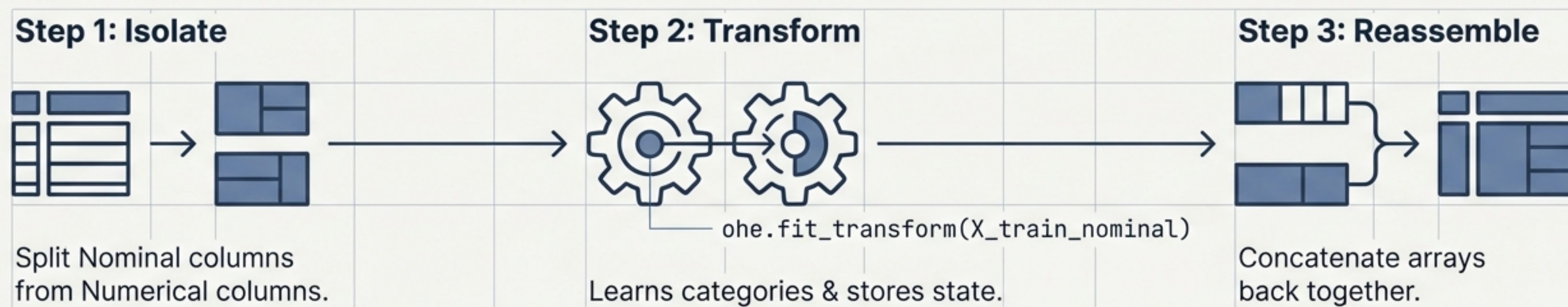
- Fast and intuitive for initial analysis.
- Automatically detects string columns.
- Returns readable DataFrame with named columns (e.g., 'Fuel_Diesel').



- **Production Risk:** Does not store encoder state.
- If training data has 'Diesel' but test data doesn't, column shapes will mismatch.
- Breaks Machine Learning Pipelines.

Implementation: The Scikit-Learn Approach

Robust, Reproducible, Pipeline-Ready



```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(drop='first', sparse=False)
X_train_new = ohe.fit_transform(X_train[['Fuel', 'Owner']])
```

Key Advantage: Guarantees consistent output structure for future data, **preventing production crashes.**

Scikit-Learn Configuration Details

- `drop='first'`
 - Crucial for linear models. Automatically removes the first category to avoid the Dummy Variable Trap (Multicollinearity).
- `sparse=False`
 - Returns a dense NumPy array instead of a sparse matrix. Useful for debugging and viewing data, though uses more memory.
- `dtype=np.int32`
 - Forces output to integer (0/1) instead of float. Optimized for memory usage.



Note on Output:

Scikit-Learn returns a NumPy array, stripping column names. You must track column order manually or use advanced ``ColumnTransformer`` methods.

Best Practices Summary

- ✓ **Identify:** Ensure data is Nominal (Colour, City), not Ordinal (Size, Rank).
- ✓ **Encode:** Use One Hot Encoding to create binary vectors.
- ✓ **Sanitize:** Always use `drop='first'` to strictly follow the N-1 rule and prevent Multicollinearity.
- ✓ **Optimize:** Handle high cardinality by grouping rare categories into an 'Other' bucket before encoding.
- ✓ **Deploy:** Prefer Scikit-Learn's `OneHotEncoder` for production pipelines over Pandas `get_dummies`.

Clean data is the prerequisite for accurate models.